Assignment 4: Pipelining + Bells + Whistles

> **Info**
>
> Assignment 4 involves **1 compulsory task** and the rest is **open-ended**.
>
> Demonstrate Post-synthesis simulation **AND** FPGA hardware incorporating all the mandatory and open-ended parts. Use a single assembly program that can be entirely compiler-generated, entirely handcrafted, or a combination of the two. Let your imagination run wild!
>
> While the instructions are reasonably complete, there could still be minor updates, which will be highlighted.

Compulsory Task: Pipelining our CPU [13 points]

Implement pipelining. Pipelining should be done such that the processor supports all the requirements for Assignment 2 and Assignment 3.

The purpose of pipelining is to increase the clock frequency. You should try increasing the clock frequency, following the tip below. For the compulsory task, it is sufficient to test and report the max frequency via CLK_DIV_BITS. There will not be additional marks for the compulsory task based on the achieved clock frequency - any attempt to increase/optimize this will be an open-ended enhancement.

Hazard hardware (forwarding, stalling, flushing, etc.) is not a compulsory task. As long as your design works with an appropriate number of `nop` s, the requirements for this task can be met.

Tips for implementing pipelining

- The WE of the PC given in the templates is active high, while the one given in the pipelined design (Chapter 5 (v2)) is active low. You might want to change the PC WE in the templates to active low to be consistent.
- Even though the pipeline registers are shown as big registers storing a lot of stuff, each data stored can be thought of as being in a separate register. In other words, in your HDL code, you don't have to try and 'collect' all the bits to form a single register entity.

Systematic procedure for Pipelining

Follow these steps to implement pipelining.

1. Start by inserting the appropriate suffixes (F/D/E/M/W) for each wire, port, register, and signal. Refer to Chapter 5 (v2) for this. For every signal that goes through multiple pipeline stages, it will need to be split into two signals, separated by a pipeline register. **Note:** Generally, the signals going into every component will have the same suffix, since every component is part of one and only one pipeline stage. There are some exceptions, such as the write related signals going into register file.
2. Make the appropriate datapath connection(s) for every signal going through a pipeline register. Some connections were made implicitly, such as ALUControl. These connections will need to be made explicitly. A multiplexer must be inserted to select between `PC_F` and `PC_E`, controlled by `PCSrc_E`. Remember to change the datatype from `wire` to `reg` where necessary.
   - **Verilog**: Use a combinational always block with non-blocking assignments. For example:

```
always @(*) begin
    ExtImm_E <= ExtImm_D;
end
```

   - **VHDL**: Use a concurrent statement (**NOT** inslude a clocked process for now). For example, `ExtImm_E <= ExtImm_D;` in RV.vhd
3. Verify that your design works **EXACTLY as it did previously**, without any changes to the assembly language program. We have NOT yet inferred any registers, so there should not be any pipelining going on (yet!). If something has broken, now is a good time to debug it.
4. Now, we implement the pipeline registers.
   - **VHDL**: Move all the signals that are supposed to go through a particular pipeline register into one clocked process in the microarchitecture (everything in one big clocked process is fine too, but it is better to keep it in separate processes for better organization).
   - **Verilog**: Change the combinational `always @(*)` to `always @(posedge clk)`.

5. Initialize all your signals and registers to zero. Add a condition that sets all these signals and registers to zero when RESET is asserted, and otherwise, assigns the RHS to the LHS at the clock edge.
6. We suggest using the same clock edge for the Register file and pipeline registers, like how it is in
7. Your processor should now be pipelined, your old program may not work as it did. You will need to add `nop`s wherever there is any kind of hazard, to avoid these. For a start, just insert `nop`s such that each pair of instructions having a data hazard is spaced by at least 2 instructions, for example, insert 3 `nop`s if the two instructions are consecutive (assuming the same clock edge is used for regsiter file and pipeline registers). After each branch, insert 2 `nop`s (or more if necessary, depending on which stage the branch is taken).
8. Verify that your design works as it used to (it will just be slower because of all the `nop`s).

Increasing the clock speed

Now that you have pipelined your processor, you might be able to run it at fairly high clock speeds, especially if your program memory and data memory are small.

However, note that even with the standard 5-stage pipeline, 100 MHz (`CLK_DIV_BITS` = 0) may not always be achievable, especially if your memory size is large.

Up to ~430 MHz is possible, though very unlikely with a 5-stage pipeline. To increase the clock beyond 100 MHz or to attempt frequencies other than those achievable via CLK_DIV_BITS, you will need to make use of the FPGA's built-in clocking resource called MMCM (which uses phase-locked loops). This can be configured using a clocking wizard (Google for more info).

Tips for faster clock speeds:

- Identify the bottleneck by looking at the timing report.
- Use smaller memories.
- For larger memories, use block ram templates, and allow at least 2 clock edges, and maybe even 3 for a read. This, of course, is a major design change as the pipeline is no longer 5-stage.
- Use a hierarchy of memories.
- If the bottleneck is the execute stage, try to make it shorter by moving the PC logic to the M stage.

Now, how do we know what frequency it can run at? You set the clock frequency via CLK_DIV_BITS or Clocking Wizard, do synthesis and then implementation, and see the warnings. You'll get a critical warning after place and route that some timing constraints are not met if the frequency you set cannot be obtained. If you get this warning, the functionality is not reliable even if it runs on the board; frequency at which you do not get the warning is the maximum safe frequency.

Open-ended Enhancement [12 points]

This is the fun part - implementing performance enhancements of your choice. There is no fixed requirement to get these points, and while we suggest some enhancements below, it doesn't mean you need to implement all (or any!) of these. Just one significant performance enhancement will suffice, and this need not be limited to the ones we have listed below. The enhancement should be over and above the compulsory task - do not implement a separate design that does not have pipelining, but has enhancements.

Hazard Hardware

This is a low-hanging fruit. It is not a very substantial enhancement in itself, but you will get *some* marks. Follow the design in Chapter 5 (v2). Do it systematically and incrementally, one hazard at a time, and testing via simulation at each step.

Idea: Additional instructions

You may choose to implement some additional instructions for your processor for your enhancement, or features of instructions that have not been fully implemented. The score you get for this will depend on how much additional logic you need to add for these features.

Implement half-word/byte load and store

You might want to add support for byte and half-word load and store: `lb`, `lh`, `lbu`, `lhu`, `sb`, `sbu`. This can be useful for data memory, all read-only peripherals that have meaningful bytes/half-words, and for 7-segment display.

`ReadData_in` is the whole word that contains the word/half-word/byte you want. You need to extract what you want, with a sign/zero(`u`) extension as required by the instruction. For example, when running `lbu` (load byte unsigned) instruction, if the last 2 bits of the address is `2'b01`, and the address location specified in the instruction has `8'hAB`, `ReadData_in` is `32'hxxxxABxx`. `ReadData`, the word to be written into the destination register is `32'h000000AB` (0s as MSBs as it is `lbu`).
For `lb`, `ReadData_in[15]` should be replicated to the 24 MSBs. You have to do this conversion. This is better done in the W stage in a pipelined processor.

`WriteData_out` is a word, with word/byte/half-word aligned to where you wish to write it to within the word. The `MemWrite_out` bits of every byte to be modified should be 1. For example, when running `sb` (store byte) instruction, if the last 2 bits of the address is `2'b10` and the byte to be written is `8'hAB` (or `32'b000000AB`), `WriteData_out` should be `32'hxxABxxxx` and `MemWrite_out` should be `4'h0100` .You have to do this conversion.
Another example: when running `sh` (store halfword), if the last 2 bits of the address is `2'b10` and the half-word to be written is `16'hABCD` (or `32'h0000ABCD`), `WriteData_out` should be `32'hABCDxxxx` and `MemWrite_out` should be `4'h1100`. You have to do this conversion.

CAUTION: Unaligned data reads and writes are NOT supported. If the instruction is `lh` / `lhu` / `sh` (load/store halfword), the data memory address should be divisible by 2 (the last bit should be 0) If the instruction is `lw` / `sw`, the data memory address should be divisible by 4 (the last two bits should be 0s)

Please read the relevant comments in Wrapper.v carefully.

Potential enhancement: Unaligned requests can be detected and used to generate interrupts by editing the wrapper. This interrupt could be used to do a software emulation of unaligned access via aligned access.

RV: Implement ISA extensions

See the RISC-V Specification for ideas. Floating-point, vector and cryptography are potentially interesting ISA extensions.

Idea: Exception handling and interrupt support

You can implement some simple interrupt handling. There is no need to conform to the RISC-V interrupt architecture.

External interrupt generation logic can be done inside the wrapper fairly easily. For example, if you want an interrupt to be raised when any of the pushbuttons are pressed, you can do interrupt = `PB[3] | PB[2] | PB[1] | PB[0]`. This interrupt can then be fed into the RV module from the wrapper.

Other possible exception/interrupt sources are:

- other peripherals - e.g., counter from Wrapper
- invalid memory address - `bad_MEM_addr` from the address decoder of Wrapper
- unaligned address - easy to detect in Wrapper or RV. The additional instructions page has some info on it
- illegal instruction - from your instruction decoder
- division by zero - from MCycle

Some other tips:

- You can have a hardcoded exception handler address input to the multiplexer controlling the PC input. The interrupt input itself can be used as (part of the) multiplexer select input.
- You can write your interrupt service routine in your assembly code, figure out the starting address, and use this value as a hardcoded input to the mux. An alternative is to decide on a fixed handler address, and fill up spaces/ `nop` s in your code until the handler starts address. For example, if you fix the handler starting address to be `IROM_BASE` +0x100 and your 'main' program contains 30 instructions, you will need to add 34 `nop` s before the first instruction in the handler code so that handler code will indeed be at `IROM_BASE` +0x100.
- The address of the instruction that caused the exception/ that was about to be executed when an interrupt came in is saved into *epc*^ (exception program counter, and not `ra`). Since we aren't aiming for compliance, saving automatically to another general-purpose register that isn't used in your program is an option too. Saving to `ra` can be problematic when you have function calls as `ra` gets overwritten. You can then use `jalr` aka `jr` to return from an interrupt.

^ Note that *epc* is not a register that can be used like a general-purpose register. You need `csrr` to move its content to a general-purpose register. * If you wish to have vectored interrupts, you need to associate a number with each interrupt. You need a table/ROM with the starting addresses, which will be indexed by the interrupt number. The output of this ROM is to be fed into the PC multiplexer. The PC multiplexer control signal can be the logical OR of all the interrupt lines.

Idea: Branch prediction

Branch prediction is a cool feature you can implement. There are many ways to approach this; a simple scheme is explored in the lecture. More complex and advanced ones are out there, but we will not teach you about them; they're for you to discover for yourself.

Do not be afraid to go and read some research papers on branch prediction schemes. Trying to implement something from a paper that has been published can be a very rewarding and educational exercise.

Idea: Use other hardware peripherals

This is not an improvement in and of itself; however, implementing more peripherals in your design might help you better demonstrate the features of your processor.

For example, the Nexys 4 has an onboard microphone - perhaps that can be used to demonstrate DSP capabilities? RGB LEDs? VGA port? USB host? The world is your oyster.

Digilent's GitHub repository for the Nexys 4/4-DDR/A7 contains some useful code to help implement these peripherals.

Design Instructions

- All 32-bit combinational arithmetic and logical operations have to be performed inside the ALU or MCycle unit. *Exceptions*: `+` for calculating next `PC`, multiplication, division, enhancements such as branch prediction. Pipeline hazard detection will require comparisons (for equality only) too, but they are on 5-bit values.
- In the ALU, DO NOT use additional `+` signs -> this could infer additional adders. The existing addition framework should be good enough.
- All operators are permitted on 32-bit values outside the RV module. For example, you will have to do 32-bit comparisons in the wrapper for address decoding.
- Use of arithmetic operators:
  a. Do not use `*` operator. It is synthesizable, but in this course, we are implementing multi-cycle multiplication. `/` is not synthesizable, except on constants.
  b. All operators (including `**`, `*`, `/`, `sll`, `rem`, `mod` etc.) are allowed on constants (operations on constants are done at synthesis time, and will not infer any hardware).

Demo Program Requirements

As before, you must write an assembly program to demonstrate the functionality you have implemented. The program must satisfy these non-negotiable requirements:

1. The program must use at least one input and one output. Programs that simply produce outputs with no response to inputs are strictly **NOT ALLOWED**.
2. The program must have some useful or interesting function, and **MUST NOT be simply a testbench that produces a pass/fail message** to "prove" that functions work. Such programs are great for testing, but very bad for demonstration.
3. The program must demonstrate convincingly the specific enhancements you have made. Ideally, it should be something that would either not run at all, or run in an inferior fashion (e.g. slower) on your previously built CPU.
4. The program must not be trivial or boring.

If your program fails to meet any of the above requirements, your demonstration will be considered unconvincing. In this case, we cannot necessarily know that your CPU actually does the things you say it does. Showing a waveform in a simulation is NOT a convincing demonstration of functionality (you can't sell a waveform to a customer buying a PC, right?). We may award as low as 0 marks for an enhancement if it is not sufficiently demonstrated with a program. Please take this part seriously and craft something interesting.

Submission Info

- Assignment 4 will be evaluated in **Week 12**. The presentation schedule can be found on Canvas.
- Include
- **.v/vhd** files you have created/modified [RTL Sources, Testbench(es)]
- **.c/h** files used to generate assembly
- **.bit** files
- **.s/.asm** files (assembly programs)
- **.ppt** file - 3 to 6 slides showing performance enhancement techniques you have implemented.

in an archive with the filename **GroupXX_Monday/Friday_Lab4.zip** (replace XX with your group number) and upload it to Canvas. One submission per group is sufficient – if there are multiple submissions, the file with the latest timestamp will be taken as the final submission. ***Do not*** zip and upload the complete project folder – only those files mentioned above should be included. **The files should be the exact same files that you**

used for the demo.